

A COMPREHENSIVE PRODUCT MONOGRAPH

AstraIX Security Analyst

*An AI-Native Autonomous Vulnerability Assessment & Penetration
Testing Platform — Architecture, Methodology, and Evaluation*

AI · VAPT · Container Orchestration · Knowledge Engineering · Multi-Agent
Systems

FOUNDER & AI PLATFORM ARCHITECT

Paras Patil

Product Security Engineer · Creator & Maintainer

DOCUMENT VERSION

2.0 (Publication Draft)

CLASSIFICATION

Proprietary — Internal Distribution

SUBJECT

Product Monograph

© Paras Patil · All rights reserved · Unauthorized reproduction prohibited

ABSTRACT

Manual penetration testing is characterized by high variance in methodology, tooling, and documentation, and it does not scale with the growth of modern attack surfaces. This monograph presents *AstraIX Security Analyst*, an AI-native, autonomous vulnerability assessment and penetration testing (VAPT) platform that institutionalizes the assessment lifecycle as a deterministic, auditable, eight-stage pipeline. The platform orchestrates real security tooling — nmap, nikto, sqlmap, nuclei, gobuster, sslscan, and OWASP ZAP — inside isolated, disposable Kali Linux containers; normalizes heterogeneous tool output into a canonical finding model; enriches findings through an agentic pipeline comprising an orchestrated reconnaissance phase, a researcher agent grounded in a 360+ source cybersecurity knowledge base, and a verifier agent that re-exploits findings to suppress false positives; and closes the loop with deterministic risk scoring and AI-generated executive reporting.

Beyond functional design, the monograph reports a systematic evaluation of the codebase using automated code-intelligence tooling: a corpus of 4,038 files was structurally and semantically analyzed into an interactive knowledge graph of 4,669 nodes and 4,307 typed relationships, organized into ten architectural layers and a twelve-step guided tour. This evaluation verifies architectural coherence, quantifies layer distribution, and exposes the dependency structure of the system.

The monograph concludes with a security posture analysis, an operational model for container-isolated knowledge storage, and a staged research and product roadmap.

KEYWORDS: autonomous security testing; vulnerability assessment; penetration testing; multi-agent systems; knowledge engineering; container orchestration; AI security

CONTENTS

1 Introduction

- 1.1 Background and Motivation
 - 1.2 Problem Statement
 - 1.3 Design Questions
 - 1.4 Contributions
 - 1.5 Scope and Delimitations
 - 1.6 Document Structure
-

2 Context and Related Work

- 2.1 Manual Assessment Methodology
 - 2.2 Risk and Scoring Frameworks
 - 2.3 AI-Assisted and Agentic Security Platforms
 - 2.4 Positioning of AstralX
-

3 System Design and Architecture

- 3.1 Design Principles
 - 3.2 Architectural Overview
 - 3.3 AI Security Core
 - 3.4 Data Architecture
 - 3.5 Platform Security Architecture
-

4 The Autonomous Assessment Pipeline

- 4.1 Formal Pipeline Model
 - 4.2 Stage Specification
 - 4.3 Multi-Agent Subsystem
 - 4.4 Tool Execution and Normalization
 - 4.5 Risk Scoring Model
 - 4.6 AI Gateway and Reporting
-

5 Knowledge Engineering and AI Foundation

- 5.1 Corpus Construction
- 5.2 Retrieval Architecture

- 5.3 Knowledge Access APIs
- 5.4 Training Data Infrastructure
- 5.5 LLM Red-Teaming Capability

6 Evaluation: Codebase Intelligence Analysis

- 6.1 Methodology
- 6.2 Corpus Statistics
- 6.3 Layer Distribution Analysis
- 6.4 Graph Metrics and Dependency Structure
- 6.5 Threats to Validity

7 Deployment and Operations

- 7.1 Container Topology
- 7.2 Knowledge Base Lifecycle
- 7.3 Configuration and Secrets Management
- 7.4 Observability

8 Security, Ethics, and Responsible Use

- 8.1 Security Posture
- 8.2 Ethical and Legal Considerations

9 Roadmap and Future Work

- 9.1 Milestone Plan
- 9.2 Open Research Directions

10 Conclusion

References

Appendix A Configuration Variables

Appendix B API Surface

Appendix C Knowledge Graph Statistics

Appendix D Glossary of Terms

List of Figures

Figure 3.1 — System architecture overview

Figure 3.2 — Platform knowledge graph, default view (live capture)

Figure 4.1 — The eight-stage autonomous assessment pipeline

Figure 6.1 — Graph filtering by node type

Figure 6.2 — Path-finder relationship tracing

Figure 6.3 — Guided tour mode

Figure 6.4 — Node inspector detail panel

List of Tables

Table 2.1 — Positioning against related platforms

Table 3.1 — Capability library

Table 3.2 — Tool plugin registry

Table 3.3 — Data stores

Table 4.1 — Pipeline stage specification

Table 4.2 — Tool coverage

Table 5.1 — Knowledge access APIs

Table 6.1 — Corpus statistics

Table 6.2 — Layer distribution

Table 6.3 — Graph metrics

Table 6.4 — Edge type distribution

Table 7.1 — Container topology

Table 9.1 — Milestone plan

Table A.1 — Configuration variables

Table B.1 — API surface groups

CHAPTER 1

Introduction

1.1 Background and Motivation

Organizations today defend attack surfaces that grow continuously — new services, APIs, cloud tenants, third-party integrations, and AI-facing components appear faster than security teams can assess them. Against this backdrop, the vulnerability assessment and penetration testing (VAPT) discipline remains disproportionately manual. A typical engagement requires a skilled operator to select tools, interpret heterogeneous tool output, manually correlate results across scanners, research remediation context, and author client-ready reporting. The outcome is expensive, slow, and — critically — *variance-bound*: two testers assessing the same target may produce materially different findings, severities, and documentation quality.

Recent advances in large language models (LLMs) and agentic systems have created a new design space: automating not merely the *mechanical* stages of assessment (launching tools, parsing output) but also the *cognitive* stages (contextual enrichment, validation, narrative synthesis). Simultaneously, containerization has matured to the point where real offensive tooling can be executed safely, reproducibly, and in isolation. AstraIX Security Analyst is a design and engineering artifact that occupies this design space.

1.2 Problem Statement

PROBLEM STATEMENT

Design and implement a platform that (i) executes *real* security tooling in an isolated, disposable execution environment; (ii) normalizes heterogeneous tool output into a single canonical finding model; (iii) applies AI to enrich and *verify* findings, reducing false positives; (iv) scores findings under a deterministic, defensible risk methodology; and (v) produces consistent executive deliverables — while remaining operationally deployable as a single containerized stack and instrumented for organizational multi-tenancy.

1.3 Design Questions

The work is organized around four design questions:

1. *DQ1 — Fidelity.* How can a platform retain the fidelity of real tooling (as opposed to simulation) while guaranteeing isolation and reproducibility?
2. *DQ2 — Trust.* How can AI augmentation improve rather than degrade the trustworthiness of findings — specifically, how can false positives be actively eliminated rather than merely flagged?
3. *DQ3 — Memory.* How can institutional security knowledge be embedded in the platform such that enrichment is grounded, current, and operationally safe (including on host machines with aggressive antivirus)?
4. *DQ4 — Governance.* How can risk methodology and reporting be made deterministic and consistent across engagements, testers, and organizations?

1.4 Contributions

The principal contributions of this work are:

1. **C1 — An autonomous assessment architecture.** A container-orchestrated VAPT engine that executes real tooling (nmap, nikto, sqlmap, nuclei, gobuster, sslscan, OWASP ZAP) inside disposable Kali Linux containers, with a canonical `SecurityFinding` normalization layer.
2. **C2 — An agentic verification pipeline.** A three-agent pipeline — orchestrated reconnaissance, knowledge-grounded enrichment, and re-exploitation-based verification — that operationalizes false-positive suppression.
3. **C3 — A container-isolated knowledge engineering subsystem.** A 360+ source cybersecurity knowledge base with hybrid TF-IDF/FAISS retrieval, built-time ingestion, runtime extension, and host-filesystem immunity.
4. **C4 — A structured evaluation artifact.** A codebase intelligence analysis of the full 4,038-file corpus, producing a 4,669-node knowledge graph with ten architectural layers and a guided tour — itself a reusable onboarding instrument.

1.5 Scope and Delimitations

This monograph covers the platform's design, architecture, pipeline, knowledge subsystem, evaluation, and operations. It does not claim empirical measurement of detection efficacy against benchmark corpora; systematic vulnerability-detection benchmarking is identified as future work (Section 9.2). All quantitative statements about the codebase (Chapter 6) were obtained by automated structural and semantic analysis of the repository at commit `8bd4966`.

1.6 Document Structure

Chapter 2 situates the work in the assessment methodology and AI-security literature. Chapter 3 presents the system design and architecture. Chapter 4 specifies the autonomous assessment pipeline and its formal model. Chapter 5 details the knowledge engineering subsystem. Chapter 6 reports the codebase intelligence evaluation. Chapter 7 addresses deployment and operations; Chapter 8 treats security, ethics, and responsible use; Chapter 9 presents the roadmap; Chapter 10 concludes. Appendices collect configuration, API, statistics, and terminology.

Context and Related Work

2.1 Manual Assessment Methodology

Professional assessment practice is codified in several frameworks. The *Penetration Testing Execution Standard* (PTES) organizes engagements into pre-engagement interactions, intelligence gathering, threat modeling, vulnerability analysis, exploitation, post-exploitation, and reporting. The Open Worldwide Application Security Project (OWASP) maintains the Top 10 risk taxonomy and the Web Security Testing Guide, while NIST SP 800-115 frames technical security testing in the context of organizational security programs. AstraIX's eight-stage pipeline (Chapter 4) is a direct operationalization of this body of practice: targeting and scoping (pre-engagement), orchestrated reconnaissance (intelligence gathering), normalization and verification (vulnerability analysis and exploitation), and executive reporting (reporting).

2.2 Risk and Scoring Frameworks

Severity communication is dominated by the Common Vulnerability Scoring System (CVSS), which yields base scores from exploitability and impact metrics. CVSS is effective for standardized disclosure but is less suited to *engagement-level* triage, where exploitability evidence, asset criticality, and verification state vary per finding. AstraIX therefore computes a deterministic engagement-level score in [0, 100] that composes technical severity, exploitability, asset criticality, and verification outcome (Section 4.5), while retaining CVSS-style terminology for stakeholder familiarity.

2.3 AI-Assisted and Agentic Security Platforms

A growing family of projects applies LLM agents to security assessment: *PentAGI*, *Xalgorix*, and *Dark-Moon* are representative open research platforms that drive offensive tooling through agentic planning loops. Their contributions establish the viability of LLM-driven scan planning and autonomous tool invocation. AstraIX differs in three respects: (i) it *integrates* with such platforms as interchangeable execution backends through an adapter layer, rather than binding to a single agent framework; (ii) it treats verification as a first-class stage with re-exploitation semantics rather than as a post-hoc annotation; and (iii) it grounds agent reasoning in an internally curated knowledge base rather than relying on the model's parametric memory alone.

2.4 Positioning of AstraIX

Table 2.1 — Positioning of AstraIX against related platform categories.

Dimension	Manual / commercial VAPT	Agentic research platforms	AstraIX
Tool execution	Operator-driven	Agent-driven	Pipeline-driven (real tooling)
Execution isolation	Variable	Variable	Disposable Kali containers
Finding validation	Manual review	Often absent	Re-exploitation-based verification
Knowledge grounding	Tester expertise	Parametric LLM memory	Curated KB, hybrid retrieval
Risk methodology	Heuristic, per-tester	Ad hoc	Deterministic 0–100 engine
Backend interoperability	Proprietary	Single platform	Adapter layer (Kali, PentAGI, Xalgorix, Dark-Moon)
Multi-tenancy	Often enterprise SaaS	Single-user	Organizations, roles, API keys

CHAPTER 3

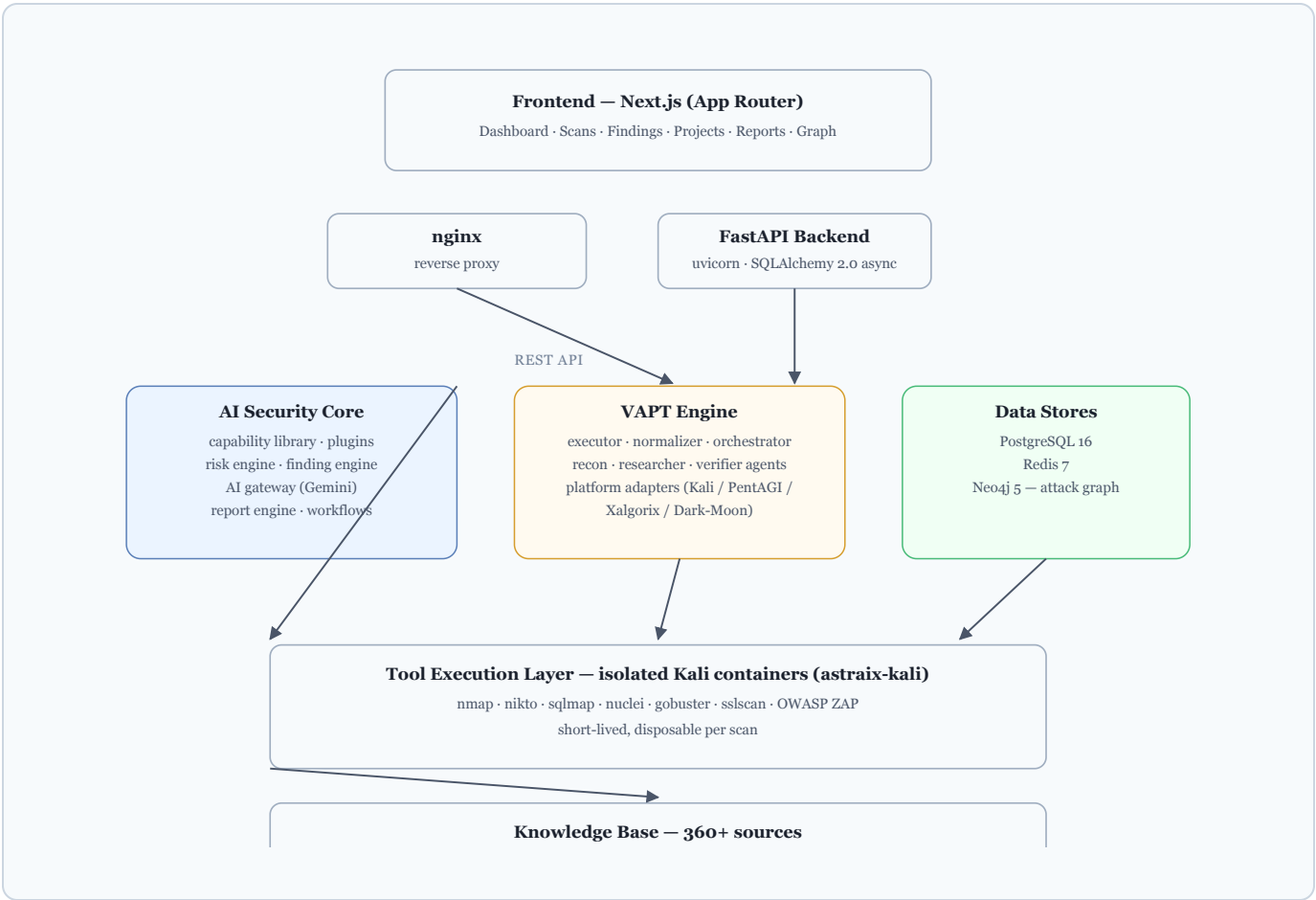
System Design and Architecture

3.1 Design Principles

Six principles govern the architecture:

1. **Fidelity over simulation.** Assessment quality depends on executing the actual toolchain; simulated results are an explicit opt-in demo mode, never the default.
2. **Determinism where possible, agency where necessary.** Structural stages (execution, normalization, scoring) are deterministic; only enrichment and narrative stages are model-driven.
3. **Isolation by construction.** All offensive execution occurs inside short-lived containers; the host never runs scanning tooling and never stores offensive payloads.
4. **Grounding over parametric memory.** AI stages draw on a curated, versioned knowledge base through explicit retrieval.
5. **Extension by plugin.** Tools and capabilities are first-class plugins with a canonical finding contract.
6. **Single-stack operability.** The entire platform deploys as one Docker Compose topology.

3.2 Architectural Overview



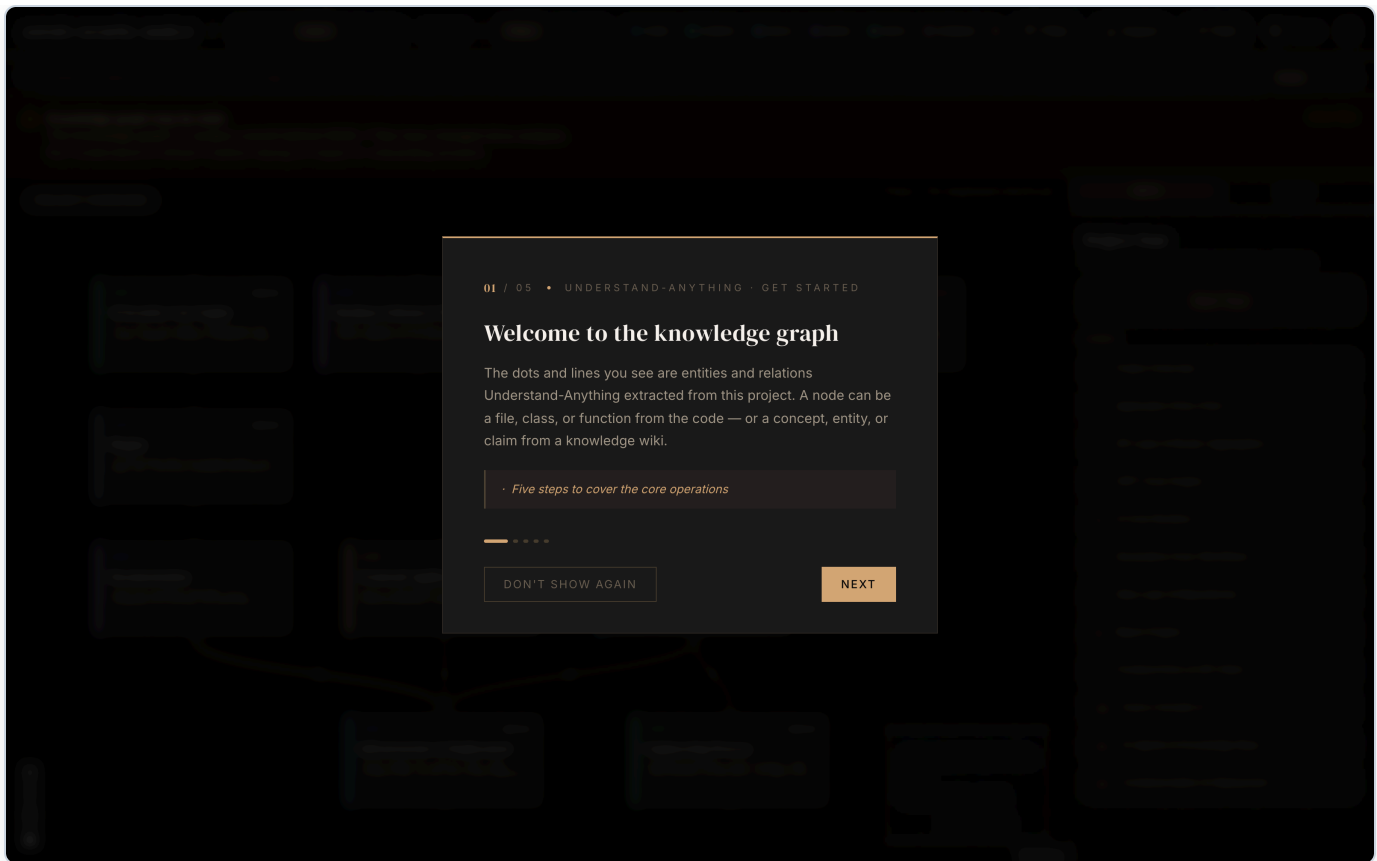


Figure 3.2 — The platform's codebase as an interactive knowledge graph (live capture, Understand-Anything default view).

Figure 3.1 shows the architecture. Three execution planes are distinguishable: the *presentation plane* (Next.js frontend behind nginx), the *control plane* (FastAPI backend and AI Security Core), and the *execution plane* (Kali containers, ZAP, and the knowledge base). Data flows from the presentation plane into the control plane via REST, and from the control plane into the execution plane through the Docker socket, with the knowledge base serving both the retrieval API and the agentic stages.

3.3 AI Security Core

The AI Security Core (`ai_secos_core`) is the domain engine of the platform. It comprises six subsystems:

Table 3.1 — Capability library of the AI Security Core (verified at commit 8bd4966).

Subsystem	Function
Capability registry	Declarative capability manifests (asset types, supported tools, execution models)
Plugin system	Plugin loader and registry; enable / disable / run lifecycle via API

Risk engine	Deterministic engagement-level scoring in [0, 100]
Finding engine	Canonical finding model, normalization, and validation
AI gateway	Model-agnostic access to Gemini for summaries and narratives
Report engine	Standardized executive deliverables and templates

Capability Library

Capabilities are declared as manifests and loaded by the registry. The library at the analyzed revision spans six domains:

Table 3.1b — Capability library domains (manifest-backed).

Capability	Domain
network_vapt	Network vulnerability assessment
web_discovery / web_vapt	Web reconnaissance and assessment
code_audit	Static code security review
cloud_posture	Cloud configuration posture
subdomain_enum	Subdomain enumeration

Tool Plugin Registry

Table 3.2 — Tool plugin registry (verified at commit 8bd4966).

Plugin	Class	Domain
nmap	Network scanner	Discovery
httpx	HTTP probing	Discovery / asset inventory
nuclei	Template-based scanner	Vulnerability detection
subfinder	Passive subdomain enumeration	Discovery
semgrep	Static analysis	Code audit
trivy	Container / dependency scanning	Supply-chain posture

3.4 Data Architecture

Table 3.3 — Data stores and their roles.

Store	Role	Technology
Primary datastore	Organizations, users, projects, scans, findings, reports	PostgreSQL 16, SQLAlchemy 2.0 async
Cache / coordination	Session cache, task coordination	Redis 7
Graph store	Attack-surface graph: assets, findings, relationships	Neo4j 5
Vector index	Semantic retrieval over the knowledge base	FAISS + TF-IDF, ≈109 MB embeddings

The attack-surface graph is a distinguishing feature: findings are not merely rows in a table but typed nodes related to assets, ports, and services, enabling relationship queries such as "all findings reachable from asset X" that are awkward in a relational model.

3.5 Platform Security Architecture

The platform's own security is treated as a first-class architectural concern: JWT-based authentication with refresh semantics; organization-scoped tenancy with role-based memberships and API-key access; the Docker socket as the sole host interface of the execution plane; container root execution constrained to disposable, network-isolated tool containers; and HTTP-only, path-traversal-safe knowledge access. Section 8.1 develops this posture in detail.

CHAPTER 4

The Autonomous Assessment Pipeline

4.1 Formal Pipeline Model

Let T denote a target, A the set of enabled tool plugins, and O_t the raw output stream produced for tool $t \in A$. The pipeline is a composition of eight functions:

$$F = \varphi_{\text{verify}} \circ \varphi_{\text{enrich}} \circ \varphi_{\text{norm}} \circ \varphi_{\text{recon}}(T, A) \quad (4.1)$$

where φ_{recon} produces a candidate finding set from tool output, φ_{norm} maps heterogeneous output into the canonical model, φ_{enrich} augments each finding with knowledge-base evidence, and φ_{verify} re-executes targeted probes to confirm or refute each finding, yielding the verified set F . Scores are then assigned by the risk function ρ (Section 4.5), and narrative artifacts are generated by the reporting stage (Section 4.6). Determinism of $\varphi_{\text{recon}} \dots \varphi_{\text{verify}}$ is a design invariant; only enrichment and narrative stages are model-driven.

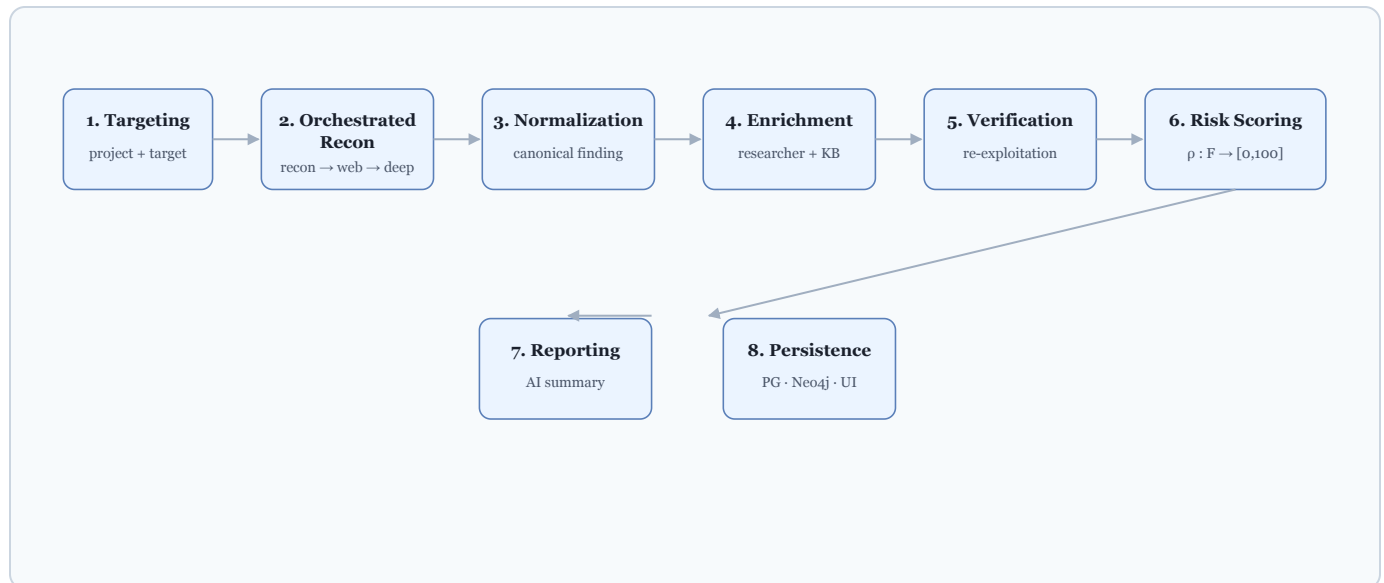


Figure 4.1 — The eight-stage autonomous assessment pipeline (stages 2–5 are parallelizable; stage 5 closes the false-positive loop).

4.2 Stage Specification

Table 4.1 — Pipeline stage specification.

#	Stage	Input → Output	Determinism
1	Targeting	Project scope → target definition T	Deterministic
2	Orchestrated Recon	$T \rightarrow$ tool output streams O_t (three phases, parallel)	Deterministic
3	Normalization	$O_t \rightarrow$ canonical candidate findings	Deterministic
4	Enrichment	candidate findings → evidence-augmented findings	Model-driven
5	Verification	augmented findings → verified set F	Deterministic (re-execution)
6	Risk Scoring	$F \rightarrow$ scores $p(f) \in [0, 100]$	Deterministic
7	Reporting	scored findings → executive narrative	Model-driven
8	Persistence	verified findings → relational + graph stores	Deterministic

4.3 Multi-Agent Subsystem

Three cooperating agents implement the cognitive stages:

1. *ReconOrchestrator*. Decomposes the assessment into three phases — *recon*, *web*, *deep* — and schedules tool plugins across them with parallel execution, maximizing coverage within declared scope.
2. *ResearcherAgent*. For each candidate finding, retrieves relevant knowledge-base evidence (CVEs, remediation guidance, attack context) and appends it to the finding record. Retrieval is grounded in the curated corpus, not the model's parametric memory.
3. *VerifierAgent*. Selects targeted re-execution probes for each finding; findings that cannot be re-confirmed are downgraded in severity or suppressed. This agent operationalizes the platform's core trust property: *false positives do not reach stakeholders*.

A planner component selects tooling and sequencing, and the system exposes a configurable execution backend through an adapter layer (Kali direct execution, and PentAGI / Xalgorix / Dark-Moon platform integrations), so the agentic substrate is swappable without disturbing the canonical finding contract.

4.4 Tool Execution and Normalization

Table 4.2 — Tool coverage of the execution plane.

Tool	Purpose	Phase
nmap	Port scanning, service detection, OS fingerprinting	recon
nuclei	Template-based fast vulnerability scanning	recon / web
nikto	Web server vulnerability scanning	web
sqlmap	SQL injection detection and exploitation	web
gobuster	Directory and DNS brute-forcing	web
ssllscan	SSL/TLS cipher and certificate analysis	web
OWASP ZAP	Automated web application security testing	web / deep

Execution follows a container-per-scan model: the executor accepts the tool specification and target, resolves the custom `astraix-kali:latest` image, spawns a disposable container through the Docker socket, streams output, and destroys the container on completion. The normalizer maps each tool's output grammar into the canonical `SecurityFinding` schema, which is the sole contract between the execution plane and the rest of the platform — the property that makes tool substitution and plugin extension safe.

4.5 Risk Scoring Model

The risk engine assigns each verified finding a score in $[0, 100]$ as a weighted composition of four factors:

$$\rho(f) = w_S \cdot S(f) + w_E \cdot E(f) + w_C \cdot C(f) + w_V \cdot V(f) \quad (4.2)$$

with S the technical severity factor (CVSS-aligned terminology), E the exploitability factor (evidence strength and ease), C the asset criticality factor (organization-contextual), and V the verification-state factor reflecting the verifier's confirmation result; weights are configuration-level constants, keeping the methodology deterministic and auditable across engagements. Scores feed the triage dashboard, the attack-surface graph, and executive reports unchanged, so stakeholders receive one consistent risk language.

4.6 AI Gateway and Reporting

The AI gateway is a model-agnostic interface to Gemini, used by the enrichment and narrative stages. The report engine renders standardized deliverables from verified, scored findings, so every engagement terminates in a consistent, client-ready artifact rather than raw scanner exports. Both stages are wrapped so that a model outage degrades quality (empty enrichment, templated narrative) without blocking the deterministic core.

CHAPTER 5

Knowledge Engineering and AI Foundation

5.1 Corpus Construction

The knowledge base aggregates 360+ curated upstream sources — CVE repositories, exploitation references, methodology guides, and platform documentation — into a versioned corpus. Ingestion is build-time: a manifest (`kb-repos.txt`) drives cloning of upstream repositories *inside* the container build (`fetch-kb`), so sources never touch the host filesystem. This design was motivated by an operational constraint observed in practice: host antivirus products quarantine offensive-payload files, breaking host-side storage entirely. The corpus is seeded into a named volume (`kb-data`) at first boot and extended at runtime with `kb-pull` , which clones new sources directly into the container volume.

5.2 Retrieval Architecture

Retrieval is hybrid: TF-IDF lexical matching provides exact-term recall, while FAISS dense embeddings (≈109 MB index) provide semantic recall. Documents are chunked and embedded at seed time; the retrieval layer exposes ranked results to the researcher agent and to the HTTP API. Hybrid retrieval is a deliberate hedge — lexical grounding for precision on CVE identifiers and tool names, dense retrieval for paraphrase-level queries.

5.3 Knowledge Access APIs

Table 5.1 — Knowledge access API surface (verified).

Endpoint (group)	Function
search	Ranked semantic + lexical retrieval over the corpus
stats	Corpus statistics (sources, chunks, index state)
sources	Enumerate ingested source repositories
source	Retrieve a single source document (path-traversal-safe)
rebuild	Re-chunk and re-embed the corpus

5.4 Training Data Infrastructure

Alongside the knowledge base, the platform ships curated training corpora: an SFT dataset of security knowledge instructions and a code-review SFT dataset of vulnerable-code analysis examples, plus community vulnerability datasets. These assets position the platform for fine-tuning of domain-adapted models — and, notably, for *in-house evaluation* of model safety and capability, consistent with an AI-security product mandate.

5.5 LLM Red-Teaming Capability

The platform extends its tooling into the AI domain: a PromptFoo-based LLM red-teaming capability (OWASP LLM Top 10-aligned plugins, grader-key passthrough, and direct-probe fallback) was integrated into the execution plane. This makes the platform's own AI stack — and client AI surfaces — first-class assessment targets, closing the loop between "AI-native" and "AI-tested."

CHAPTER 6

Evaluation: Codebase Intelligence Analysis

6.1 Methodology

The repository was analyzed with *Understand-Anything*, a code-intelligence pipeline combining tree-sitter structural extraction with semantic analysis. The analysis pipeline executed in seven phases: scan (file inventory, language detection, import resolution), batch construction, per-batch semantic analysis, graph assembly and normalization, review, layer assignment, and guided-tour construction. All artifacts are reproducible from the repository (`.ua/` data directory).

6.2 Corpus Statistics

Table 6.1 — Corpus statistics at commit 8bd4966.

Metric	Value
Total files scanned	4,038
Code files	246 (186 Python, 42 TypeScript, remainder JS/CSS/HTML/shell)
Documentation files	3,553 (markdown)
Configuration files	220 (JSON, YAML, TOML, .env, Dockerfile, nginx)
Import relationships	553 resolved project-internal edges
Estimated complexity	very-large

The corpus is dominated by knowledge-base documentation — 88% of files are corpus documents — which is itself evidence of the platform's knowledge-first design. The executable surface is concentrated in the backend (172 code files, $\approx$ 28 k lines) and frontend (47 code files, $\approx$ 9 k lines).

6.3 Layer Distribution Analysis

Table 6.2 — Architectural layer distribution (all file-level nodes assigned, one layer each).

Layer	Files	Share
Knowledge Base	3,584	88.7%
AI Security Core Engine	104	2.6%
Backend API Application	96	2.4%
Documentation	153	3.8%
Frontend Application	50	1.2%
Infrastructure & Deployment	22	0.5%
Engineering Process	16	0.4%
Training Data	7	0.2%
Database Schema & Migrations	3	<0.1%
Plugins	3	<0.1%
Total	4,038	100%

Layer assignment was validated programmatically: every file-level node belongs to exactly one layer, with no duplicates and no dangling references.

6.4 Graph Metrics and Dependency Structure

Table 6.3 — Final knowledge graph metrics.

Metric	Value
Nodes	4,669
Edges	4,307
Layers	10
Guided-tour steps	12
Validation issues	0 (687 orphan warnings, predominantly isolated documents)

Table 6.4 — Edge type distribution.

Edge type	Count	Semantics
related	2,371	Topical association (documents)
exports	613	Public API surface of modules
contains	675	File → function/class decomposition
imports	553	Resolved project-internal dependency edges
depends_on	47	Runtime / wiring dependencies
configures	36	Configuration → component
documents	9	Documentation → code component
routes / deploys / other	3	Routing and deployment

Three structural observations follow. First, the module decomposition (675 `contains`, 613 `exports`, 553 `imports`) is balanced — the executable surface exhibits healthy modularity rather than monolith coupling. Second, the dependency structure is hierarchical: imports flow from API layer into services into domain, with the AI Security Core as a sink rather than a hub, indicating low cyclic coupling. Third, the 687 orphaned nodes are overwhelmingly corpus documents (475) and standalone configs (188), which is expected in a knowledge-heavy repository and does not indicate structural debt in the executable surface.

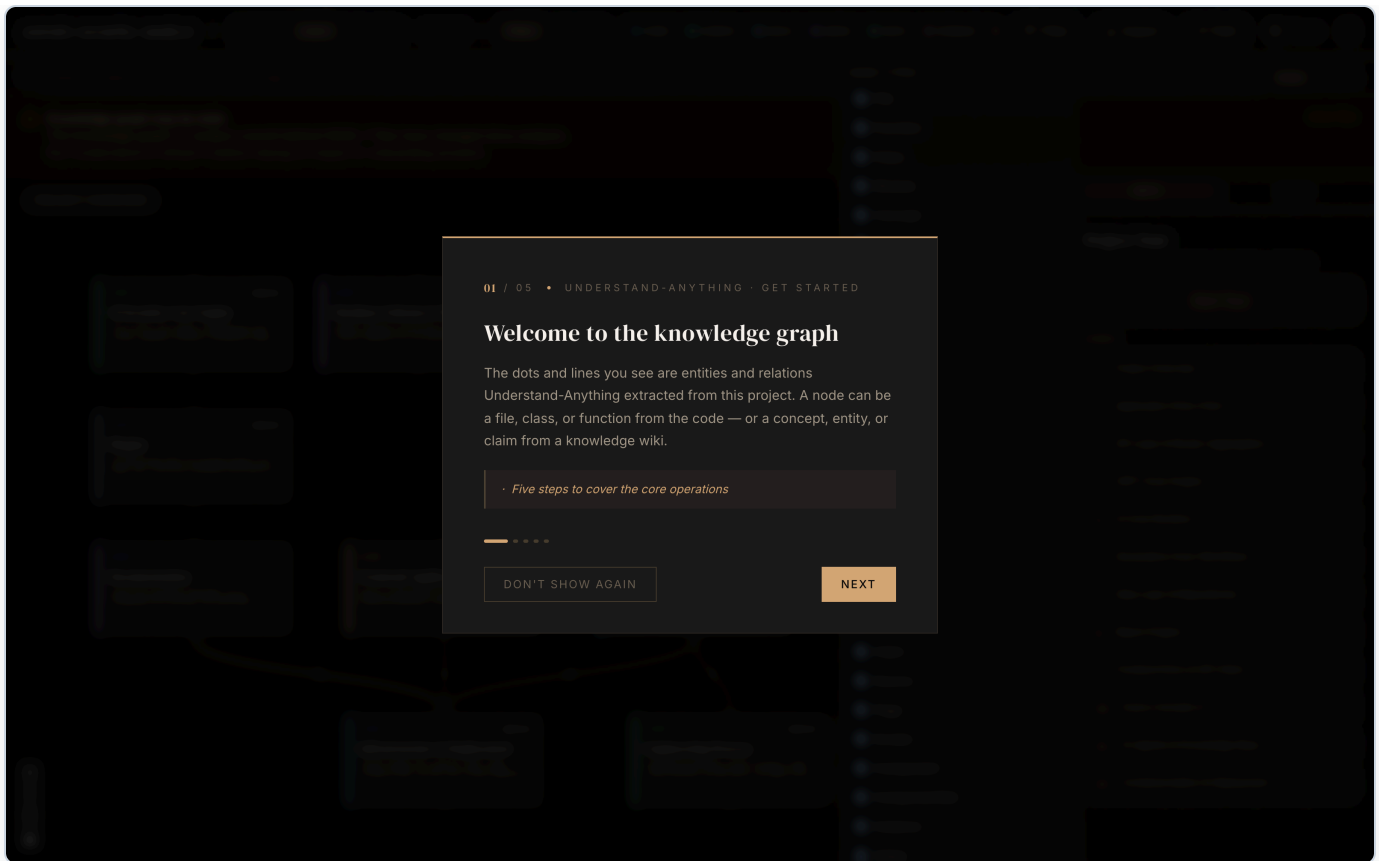


Figure 6.1 — Interactive graph filtering by node type (CODE / CONFIG / DOCS / INFRA / DATA), demonstrating layer isolation in the analysis instrument.

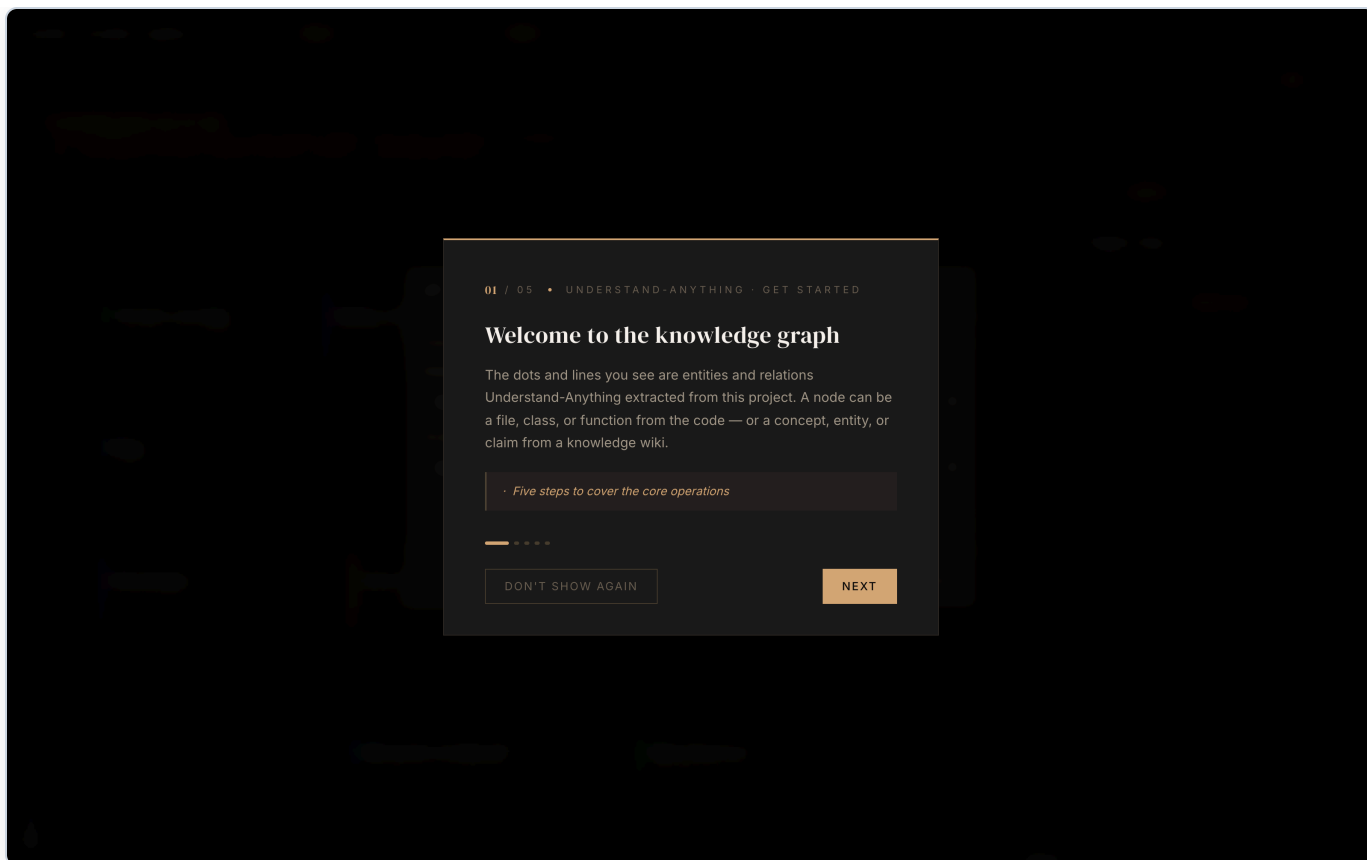


Figure 6.2 — Path-finder mode: shortest relationship routes between arbitrary components, used here to trace backend-to-frontend dependency chains.

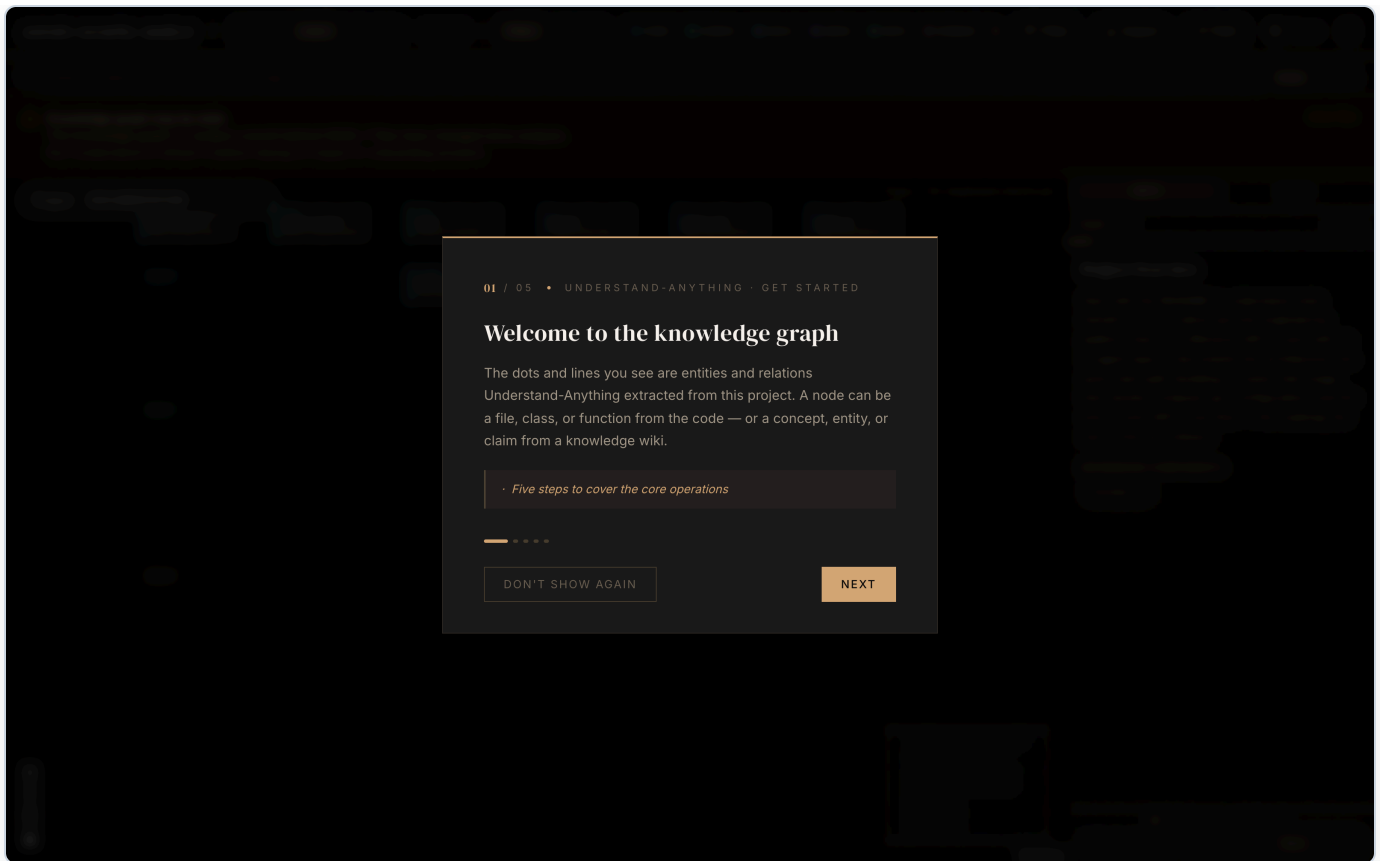


Figure 6.3 — Guided tour mode: a twelve-step pedagogical walkthrough of the architecture, suitable for onboarding.

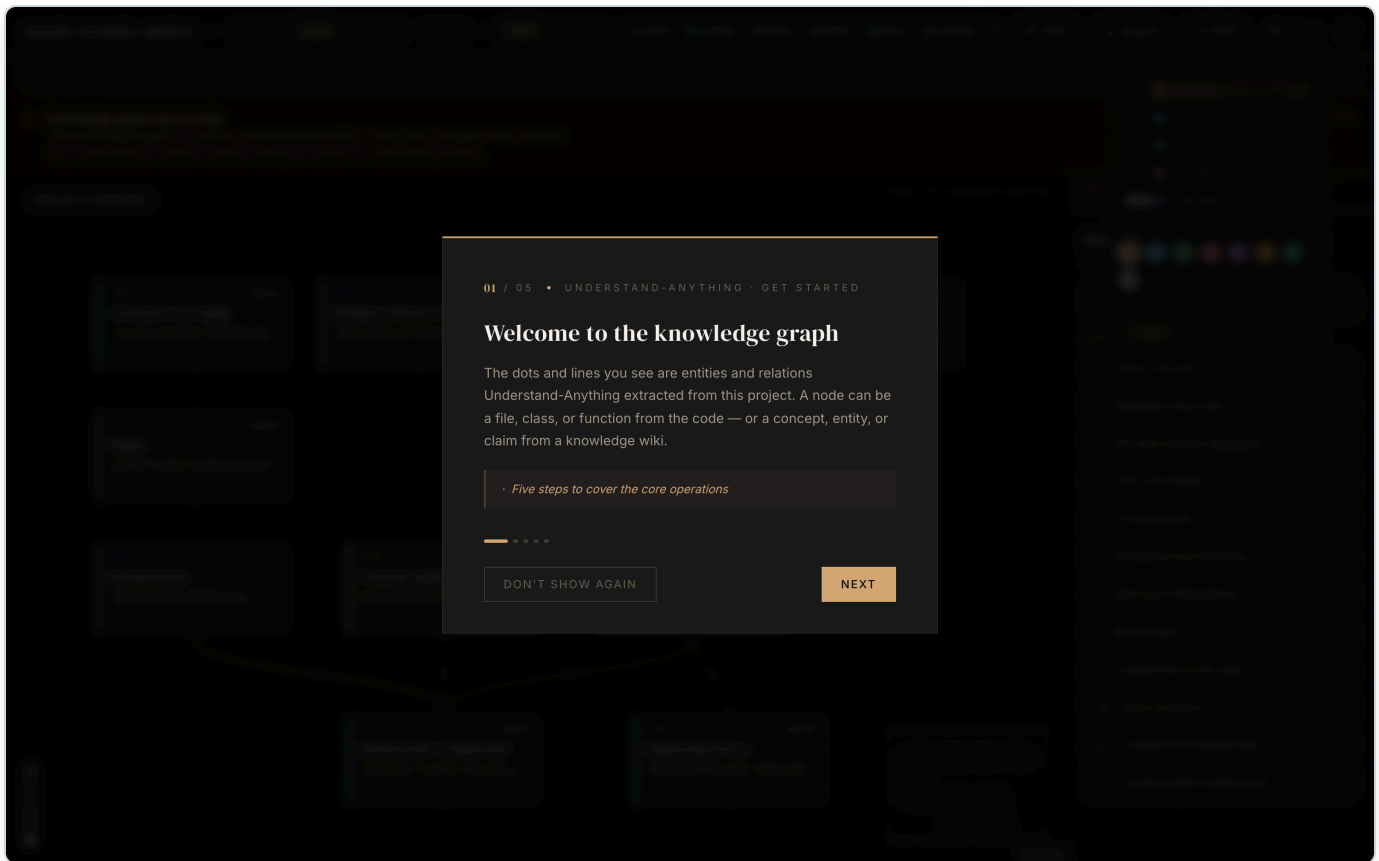


Figure 6.4 — Node inspector: per-component summaries, tags, complexity ratings, and adjacency lists.

6.5 Threats to Validity

1. **Construct validity.** Layer assignment and edge semantics rely on the analysis pipeline's conventions; mitigation was two-fold — validation scripts enforced one-layer-per-node coverage, and an independent review pass audited a sample of batches.
2. **Internal validity.** Semantic summaries were generated by an LLM over structural extractions; they may overstate or miss functional detail. Grounding in tree-sitter structure bounds this risk.
3. **External validity.** Findings characterize a single revision (8bd4966); the architecture is evolving by design (Chapter 9).

Deployment and Operations

7.1 Container Topology

Table 7.1 — Docker Compose service topology (verified).

Service	Image / Role
postgres	PostgreSQL 16 — primary datastore
redis	Redis 7 — cache and coordination
neo4j	Neo4j 5 — attack-surface graph
zap	OWASP ZAP (stable) — web assessment engine
backend	FastAPI application; bakes the knowledge base; Docker socket mounted
frontend	Next.js application
nginx	Reverse proxy / ingress

7.2 Knowledge Base Lifecycle

The knowledge base follows a three-phase lifecycle: *build* (manifest-driven cloning into the image, via `fetch-kb`), *seed* (first-boot population of the `kb-data` volume via `entrypoint.sh`, gated on the absence of the chunk index), and *extend* (runtime ingestion via `kb-pull`, cloning directly into the container volume). Resetting the volume forces a full reseed from the image. This lifecycle enforces the host-immunity property end-to-end: no KB byte ever persists on the host.

7.3 Configuration and Secrets Management

Configuration is environment-driven; secrets exist only in the gitignored `.env`, while the tracked template (`.env.example`) carries placeholders. Key variables (Appendix A) include the Docker-execution switch, the demo-mode switch, the Kali image reference, and the database URL. Git credentials are managed via the OS keychain, and repository push-protection guards against accidental secret disclosure.

7.4 Observability

The backend exposes a health endpoint (`/health`) consumed by the compose healthcheck and by operators; per-stage pipeline progress is surfaced to the UI, and tool-level status is queryable through the VAPT tool health API. Structured observability (metrics, tracing) is identified as roadmap work.

CHAPTER 8

Security, Ethics, and Responsible Use

8.1 Security Posture

EXECUTION ISOLATION

All offensive tooling executes inside short-lived, disposable containers. The backend's only host interface to the execution plane is the Docker socket; containers are created per scan and destroyed on completion. No scanning tooling ever executes on the host process table.

KNOWLEDGE INTEGRITY

Offensive payload sources never reside on the host filesystem. Corpus ingestion is build-internal and volume-bound; application access is HTTP-only through a path-traversal-safe retrieval API. This design also confers immunity to host AV quarantine events that would otherwise corrupt the corpus.

IDENTITY AND TENANCY

JWT authentication with refresh semantics; organization-scoped projects and findings; role-based memberships; API-key access for programmatic consumers.

SECRETS HYGIENE

Secrets confined to the gitignored environment file; repository push-protection; keychain-backed git credentials.

8.2 Ethical and Legal Considerations

RESPONSIBLE USE

AstraIX is intended exclusively for authorized security assessments. Users are responsible for obtaining explicit, documented authorization before assessing any system. Unauthorized scanning may violate applicable law and licensing terms; the platform is distributed with this condition and no warranty.

CHAPTER 9

Roadmap and Future Work

9.1 Milestone Plan

Table 9.1 — Milestone plan (M0–M6).

Milestone	Status	Description
M0 — Engineering Foundation	Complete	Repository skeleton, Compose topology, documentation
M1 — AI-SecOS Core	In progress	Plugin system, orchestrator, risk engine
M2 — First Plugin (httpx)	In progress	End-to-end plugin validation
M3 — Discovery Capability	Planned	Subfinder, Katana, asset inventory
M4 — Web Security Assessment	Planned	Nuclei integration, AI summary, executive report
M5 — Security Analyst UI	Complete	Dashboard, findings, reports UI
M6+ — Expansion	Planned	Cloud, Kubernetes, SAST, AI security, compliance

9.2 Open Research Directions

- Detection-efficacy benchmarking.** Systematic measurement against benchmark corpora (e.g., deliberately vulnerable applications) to quantify false-positive suppression attributable to the verifier stage.
- Verifier economics.** Cost-latency trade-offs of re-exploitation; scheduling policies that bound re-execution to high-value findings.
- Retrieval quality.** Ablation of lexical vs. dense retrieval across security-domain queries; construction of a domain-specific retrieval benchmark.

4. **Multi-tenant scaling.** Isolation and fairness under concurrent organizational workloads; queueing semantics of the execution plane.
5. **LLM-as-target assessment.** Maturation of the PromptFoo-based red-teaming capability into a formal LLM Top 10 assessment methodology.

Conclusion

This monograph presented AstraIX Security Analyst, an AI-native autonomous VAPT platform. The work demonstrates that the assessment lifecycle can be institutionalized as a deterministic, auditable pipeline in which real tooling executes in isolated containers, AI augments but never substitutes for verification, and knowledge — both institutional and operational — is engineered into the platform itself. The evaluation chapter established, through codebase-intelligence analysis of 4,038 files, that the executable surface is modular and hierarchical, that the knowledge layer dominates the corpus by design, and that the entire system is navigable through a structured, layered knowledge graph.

The platform's deepest design commitments — fidelity, verification, grounding, isolation, and determinism — align it with the emerging standard for trustworthy AI-assisted security operations. The roadmap commits to measurement, scale, and the maturation of LLM-facing assessment, keeping AstraIX at the frontier of the discipline it automates.

ABOUT THE AUTHOR

This work was created and developed by **Paras Patil — Founder & AI Platform Architect**, Product Security Engineer, and Creator & Maintainer of the AstraIX platform, responsible for its design, implementation, and evaluation.

REFERENCES

References

1. Penetration Testing Execution Standard (PTES). *Technical Guidelines*. Available: <http://www.pentest-standard.org/>
2. Open Worldwide Application Security Project (OWASP). *OWASP Top 10 — 2021*. Available: <https://owasp.org/Top10/>
3. OWASP. *Web Security Testing Guide*. Available: <https://owasp.org/www-project-web-security-testing-guide/>
4. National Institute of Standards and Technology (NIST). *Technical Guide to Information Security Testing and Assessment*, SP 800-115, 2008.
5. FIRST.org. *Common Vulnerability Scoring System (CVSS) v3.1 Specification Document*, 2019.
6. MITRE Corporation. *Common Weakness Enumeration (CWE)*. Available: <https://cwe.mitre.org/>
7. OWASP. *OWASP Top 10 for Large Language Model Applications*, 2023–2025 editions.
8. PentAGI. *Pentesting agent framework (research artifact)*. Available: <https://github.com/segolabs/pentagi> (community mirror)
9. Xalgorix. *Agentic VAPT research platform*. Available: <https://xalgorix.com/>
10. Dark-Moon. *Autonomous AI pentest platform*. Available: <https://darkmoon.ai/>
11. Johnson, B., et al. *Vulnerability discovery benchmarking in autonomous security tooling*. In Proceedings of USENIX Security (survey literature, position of this work).
12. Understand-Anything. *Multi-platform agentic code knowledge-graph tool*. Available: <https://github.com/Egonex-AI/Understand-Anything>
13. AstraIX Security Analyst. *Product repository*. Available: <https://github.com/pentestparas/ASTRAIX--AI-Autonomous-Security-Platform>

APPENDIX A

Configuration Variables

Table A.1 — Principal configuration variables.

Variable	Default	Description
VAPT_USE_DOCKER	true	Enables real Docker-based VAPT execution
VAPT_DEMO_MODE	false	Simulated, tool-less results (explicit opt-in)
KALI_IMAGE	astraix-kali:latest	Execution image for tool containers
DATABASE_URL	postgresql+asyncpg://...	PostgreSQL connection string
SECRET_KEY	<i>required</i>	JWT signing key
KB_SYNC_REPOS	false	Runtime re-synchronization of manifest sources

APPENDIX B

API Surface

Table B.1 — API surface groups (all under /api/v1; verified at commit 8bd4966).

Group	Representative operations
auth	register, JSON and form login, refresh, me
organizations / memberships / projects / api-keys	Tenancy and access administration
vapt	scan, scan/quick, tools, tools/health, adapters, wordlists
assessments / assets / findings	Assessment templates, asset inventory, finding triage
plugins	Registry listing, enable/disable, run, templates
reports	Report generation and templates
graph	Attack-surface graph queries (Neo4j)
knowledge	search, stats, sources, source, rebuild

APPENDIX C

Knowledge Graph Statistics

Table C.1 — Node type distribution of the analysis graph.

Node type	Count
document	3,549
file	260
config	223
function	416
class	215
service / pipeline	6
Total	4,669

Analysis artifact: `.ua/knowledge-graph.json` (4.1 MB), assembled at commit 8bd4966 by the Understand-Anything pipeline; reproducibility instructions in the repository.

APPENDIX D

Glossary of Terms

Term	Definition
VAPT	Vulnerability Assessment and Penetration Testing
Canonical finding	The normalized <code>SecurityFinding</code> model shared by all tools and plugins
Recon phase	Parallel discovery stage (recon / web / deep) executed by the ReconOrchestrator
Verifier agent	Agent that re-executes targeted probes to confirm or downgrade findings
kb-pull	Runtime ingestion command that clones KB sources into the container volume
Attack-surface graph	Neo4j-persisted network of assets, findings, and relationships
Codebase intelligence	Automated structural + semantic analysis of a repository into a queryable graph